502094 - Software Deployment, Operations And Maintenance

Module 8: Application Scaling, Load Balancing, and Observability

Lecturer: MSc. Mai Van Manh

Email: maivanmanh@tdtu.edu.vn

Foundations of Scaling

- What **scaling** means in software systems
- Why scaling matters for performance and reliability
- Key metrics: **latency, throughput, concurrency, resource usage**
- How to reason about **bottlenecks**

What Is Scaling?

- **Scaling**: increasing or decreasing system capacity to handle changing load
- Dimensions:
  - Number of users
  - Requests per second (RPS)
  - Data volume
- Goal:
  - Maintain acceptable **latency** and **error rate** as traffic grows
- In DevOps:
  - Scaling is a continuous engineering concern, not a one-time task

Why Scaling Matters

- Poorly scaled systems lead to:
  - Slow responses or timeouts
  - Server crashes during peak hours
  - Lost revenue and bad user experience
- Well-scaled systems:
  - Absorb traffic spikes gracefully
  - Remain responsive under load
  - Enable business growth without constant re-architecture

Key Performance Metrics

- **Latency**:
  - Time to respond to a request (e.g. 95th percentile < 300 ms)
- **Throughput**:
  - Requests per second the system can handle
- **Concurrency**:
  - Number of active requests or users at the same time
- **Resource usage**:
  - CPU, memory, disk I/O, network bandwidth
- These metrics guide **scaling decisions**

Understanding Bottlenecks

- A **bottleneck** is a component that limits overall performance:
  - CPU-bound (heavy computation)
  - I/O-bound (disk, network, database)
  - Lock/contention-bound (synchronization, queues)
- The system is only as fast as its **slowest critical path**
- Before scaling:
  - Measure where time is spent
  - Identify the real bottleneck

Inspecting Resource Bottlenecks (Example)

- On a Linux server:

```
top         # CPU and memory usage per process
iostat      # Disk I/O statistics
iftop       # Network usage by connection
vmstat      # System-wide memory and swap usage
```

- Scenario:
  - Your Node.js API shows high latency
  - `top` reveals 90% CPU on one core → CPU-bound
  - Scaling strategy:
    - Optimize code and/or add more instances

Vertical Scaling vs Horizontal Scaling

- Two fundamental ways to add capacity:
  - **Vertical scaling** (scale up)
  - **Horizontal scaling** (scale out)
- Trade-offs and real-world usage

Vertical Scaling (Scale-Up)

- Approach:
  - Upgrade to a bigger machine:
    - More CPU cores, More RAM, Faster SSD
- Advantages:
  - Simple operationally (no code changes)
  - Works well for small to medium loads
- Limitations:
  - Hardware limits (there is always a max size)
  - Downtime during resize (often reboot)
  - Single point of failure remains

Horizontal Scaling (Scale-Out)

- Approach:
  - Add more application instances:
    - More VMs
    - More containers, More Pods
- Characteristics:
  - Work is spread across many nodes
  - Fault tolerance improves (one node can fail)
- Requires:
  - **Stateless** services or careful state management
  - Load balancers and service discovery

Why Horizontal Scaling Is Preferred

- **Elasticity**:
  - Scale out during peak hours, scale in when idle
- **High availability**:
  - Failures are isolated to a subset of nodes
- **Cost efficiency**:
  - Use many small instances instead of one gigantic one
- Supports:
  - Rolling upgrades
  - Blue/green deployments
  - Canary releases

Real-World Horizontal Scaling Examples

- Kubernetes:
  - Multiple Pods per Deployment across nodes
- AWS EC2 Auto Scaling Group (ASG):
  - Adjust number of instances based on CPU or request metrics
- Docker:
  - Multiple container replicas behind Nginx or a load balancer
- In all cases:
  - A **load balancer** sits in front to distribute requests

Why Load Balancing Is Essential

- Scaling without a load balancer leads to:
  - Manual routing
  - Uneven traffic
  - Poor fault tolerance
- Load balancers are the **entry point** to scaled architectures

Scaling Without a Load Balancer

- Suppose you run 3 API instances:
  - `api-1` at `10.0.0.1:3000`
  - `api-2` at `10.0.0.2:3000`
  - `api-3` at `10.0.0.3:3000`
- Without a load balancer:
  - Frontend must know all instance addresses
  - Clients may hard-code one server and overload it
  - Failover is manual (change configs, deploy again)
- This does not scale in production

What a Load Balancer Does

- Acts as a **single, stable endpoint**:
  - Clients send traffic to `https://api.mystore.com`
- Distributes requests across multiple backend instances:
  - Round-robin or other algorithms
- Performs **health checks**:
  - Detects and removes unhealthy instances from rotation
- Enables:
  - Zero-downtime deployments
  - Blue/green and canary rollouts

Load Balancer and Fault Tolerance

- When a backend instance fails:
  - Load balancer stops sending traffic to it
  - Auto-scaling group or orchestrator replaces it
- Users continue to access:
  - Same domain name
  - Same URL paths
- From the client perspective:
  - System “just works” despite internal failures

Types of Load Balancers & Algorithms

- Load balancer types:
  - **Application, Network, Reverse proxy, DNS-based**
- Core routing algorithms:
  - **Round Robin, Least Connections, Weighted, IP Hash, Random**

L4 vs L7 Load Balancing

- **Layer 4 (L4)**:
  - Operates at `TCP`/`UDP` level
  - Decisions based on IP and port
  - Very fast, protocol-agnostic
  - Example: Network Load Balancer (NLB)
- **Layer 7 (L7)**:
  - Operates at `HTTP`/`HTTPS` level
  - Can inspect URL path, headers, cookies
  - Supports routing to different backends based on rules
  - Example: Application Load Balancer (ALB), Nginx reverse proxy

Application Load Balancer (ALB)

- Typical features:
  - HTTP/HTTPS termination
  - Path-based routing:
    - `/api/*` → API service
    - `/static/*` → CDN or static server
  - Host-based routing:
    - `api.mystore.com` vs `admin.mystore.com`
- Integration:
  - With auto-scaling groups and health checks
  - With certificates for TLS termination

Network Load Balancer (NLB)

- Works at L4:
  - Very high performance, low latency
- Suitable for:
  - TCP-based protocols
  - gRPC, custom binary protocols
- Common in:
  - High-throughput microservices
  - Systems where you do TLS termination on the backend

Reverse Proxy Load Balancer (Nginx)

- **Nginx** as reverse proxy:
  - Accepts client traffic on behalf of backend servers
  - Forwards it to upstream servers
- Advantages:
  - Rich configuration language
  - Can act as L7 load balancer, SSL terminator, and cache
- Often used:
  - In front of Dockerized backends
  - As an Ingress controller in Kubernetes

DNS-Level Load Balancing (Route 53)

- DNS services like Route 53 can:
  - Return different IPs for the same domain
  - Perform basic health checks
- Use cases:
  - Global load balancing between regions
  - Disaster recovery:
    - Route traffic away from failing region
- Limitation:
  - DNS caching and TTL delays
  - Not as responsive as in-cluster load balancers

Load Balancing Algorithms: Round Robin

- **Round Robin**:
  - Assign each incoming request to the next server in order
  - Simple and widely used
- Example (Nginx upstream):

```
upstream api_upstream {
  server api1:3000;
  server api2:3000;
  server api3:3000;
}
```

- Works well when:
  - All servers have similar capacity
  - Requests are roughly equal in cost

Weighted Round Robin

- Used when servers have different capacities
- Example:

```
upstream api_upstream {
  server api1:3000 weight=3;
  server api2:3000 weight=1;
}
```

- `api1` receives `~75%` of traffic, `api2` receives `~25%`
- Useful when:
  - Migrating traffic to a new cluster gradually
  - Handling mixed instance types (e.g., large vs small EC2)

Least Connections and IP Hash

- **Least Connections**:
  - Route new request to server with fewest active connections
  - Good when:
    - Some requests are long-lived
    - Load per request is uneven
- **IP Hash (Sticky Sessions)**:
  - Same client IP always hashes to same backend
  - Used when:
    - Session data is stored locally on backend
    - You cannot yet centralize session storage
- Over time: Modern systems avoid sticky sessions by design (stateless)

Requirements for Horizontally Scalable Web Applications

- To work well behind a load balancer, web apps must:
  - Be **stateless** as much as possible
  - Externalize shared state to proper systems

Stateless vs Stateful Applications

- **Stateful app**:
  - Stores user-specific data in memory or local disk:
    - HTTP sessions
    - In-memory caches
    - File uploads
- **Stateless app**:
  - Treats each request independently
  - Persists state to external systems:
    - Databases, Caches, Object storage
- Stateless design: Makes horizontal scaling straightforward

Session Management for Scalable Web Apps

- Avoid storing sessions in process memory:
  - If a node fails, users are logged out
  - Sticky sessions create coupling to individual nodes
- Prefer centralized session storage:
  - **Redis** session store for Node.js/Express, Spring, etc.

```
app.use(session({
  store: new RedisStore({ host: "redis", port: 6379 }),
  secret: "change-this-secret",
  resave: false,
  saveUninitialized: false
}));
```

- Now any backend instance can serve any user session

Centralized Storage for Uploaded Files

- Problem:
  - Storing user uploads on local disk (`/uploads`) of one server
- When behind a load balancer:
  - Another instance cannot see those files
- Solution:
  - Store files in shared storage:
    - Amazon S3
    - Google Cloud Storage
    - NFS or dedicated file server
- Backend only stores:
  - File metadata and URLs in the database

Distributed Caching and Real-Time Messaging

- **Caching**:
  - Use Redis or Memcached as a shared cache layer
  - All instances read/write from the same cache
- **Real-time chat (WebSockets)**:
  - Single-node chat server cannot scale horizontally
  - Use Redis Pub/Sub or similar as a message bus
  - Any instance can publish/subscribe to messages
- Result:
  - Multiple nodes behave as one logical real-time system

Externalizing Configuration

- Hard-coded configuration:
  - API URLs, feature flags, secrets embedded in code
  - Requires redeploy to change environment settings
- Better approach:
  - Use **environment variables** and config files
- Example (Docker Compose snippet):

```
environment:
  - NODE_ENV=production
  - LOG_LEVEL=info
  - REDIS_URL=redis://cache:6379
```

- Makes scaling and environment changes safer and faster

Load Balancing in Docker Compose

- Use Compose to:
  - Run multiple API replicas
  - Place Nginx in front as a load balancer
- Key ideas:
  - Only Nginx exposes public ports
  - API replicas are reachable via internal Docker network

Scaling Web API Services in Compose

```yaml
version: "3.9"
services:
  api:
    build: ./api
    deploy:
      replicas: 3
  balancer:
    image: nginx:latest
    ports:
      - "8080:80"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
    depends_on:
      - api
```

- Only `balancer` exposes port 8080 to the host

Why Replicas Should Not Expose Public Ports

- If each replica exposes a host port:
  - You must manually round-robin across ports
  - Hard to manage firewall rules
  - Attack surface expands
- Better:
  - Use internal network for replicas
  - One public entrypoint (Nginx) with hardened configuration
- Security principle:
  - **Least privilege** and minimal exposure

Nginx as a Load Balancer in Docker Compose

```
http {
  upstream api_upstream {
    server api:3000;
  }
  server {
    listen 80;
    location / {
      proxy_pass http://api_upstream;
      proxy_set_header Host $host;
      proxy_set_header X-Real-IP $remote_addr;
    }
  }
}
```

- Docker internal DNS returns multiple IPs for `api`
- Nginx balances across them (depending on resolution mode)

Switching to Weighted Load Balancing in Nginx

```
upstream api_upstream {
  server api-1:3000 weight=3;
  server api-2:3000 weight=1;
}
```

- Use Docker Compose service names that map to specific containers
- Scenario:
  - Gradually moving traffic from an old version to a new version
- Combine with:
  - Separate services `api_v1` and `api_v2`
  - Different weights for canary testing

Practical Workflow: Scaling Node.js on Local Docker

- Step-by-step:
  - Start with a single instance
  - Add replicas with Docker Compose
  - Configure Nginx to forward traffic
  - Verify distribution and resilience under failure

Step 1: Single Instance with Docker Compose

```yaml
services:
  api:
    build: ./api
    ports:
      - "3000:3000"
```

- Test:

```bash
docker compose up -d
curl http://localhost:3000/health
```

- Once verified:
  - Introduce a dedicated balancer service

Step 2: Add Nginx and Multiple API Instances

```yaml
services:
  api:
    build: ./api
    deploy:
      replicas: 3
  balancer:
    image: nginx:latest
    ports:
      - "8080:80"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
```

- Access the system via:

```
curl http://localhost:8080/api/products
```

Step 3: Demonstrate Load Distribution

- Log each API response with instance ID:

```
app.get("/api/products", (req, res) => {
  res.json({
    instance: process.env.HOSTNAME,
    products: [...]
  });
});
```

- Send multiple requests:

```
for i in {1..10}; do \
curl -s http://localhost:8080/api/products | \
jq ".instance"; \
done
```

- Observe: Different instances respond over time → load balancing

Step 4: Simulate Failure

- Stop one container:

```
docker ps
docker stop <one-api-container-id>
```

- Continue sending requests to Nginx:

```
curl http://localhost:8080/api/products
```

- If health checks and orchestration are configured:
  - Failed instance is removed
  - Remaining instances handle traffic
- Logs help investigate **when and why** instances failed

Scaling Web APIs on AWS Using EC2

- Auto Scaling Group (ASG)
- Application Load Balancer (ALB)
- End-to-end flow from deployment to auto-healing

Auto Scaling Groups (ASG) Basics

- ASG defines:
  - Launch template (AMI, instance type, security groups)
  - Minimum, maximum, and desired capacity
- Auto-healing:
  - If an instance fails health checks:
    - ASG replaces it automatically
- Scaling policies:
  - Target CPU utilization
  - Request count per instance
  - Custom CloudWatch metrics

Application Load Balancer with ASG

- ALB:
  - Listens on HTTP/HTTPS ports (80, 443)
  - Routes to **target groups**
- Target group:
  - List of EC2 instances or IPs
  - Health check configuration:
    - Path: `/health`
    - Interval and thresholds
- ALB + ASG provide:
  - Scaled, resilient API endpoint at `https://api.mystore.com`

End-to-End Flow on AWS

1. Deploy your API as an AMI or through a startup script
2. Create a Launch Template referencing the AMI
3. Create an ASG using the Launch Template
4. Attach ASG to an ALB target group
5. Configure scaling policies (e.g., CPU > 60% for 5 minutes)
6. Observe:
    - New instances added when traffic increases
    - Old instances removed when traffic drops

Database Scaling – Why It Is Harder

- Web APIs:
  - Stateless, easily replicated
- Databases:
  - Stateful, storing durable data
- Scaling databases requires:
  - Careful design of consistency, replication, and failover

SQL Database Scaling Basics

- Start with a single primary:
  - All reads and writes go to one instance
- Add read replicas:
  - Primary handles writes
  - Replicas handle read-only queries
- Challenges:
  - Replication lag
  - Failover when primary fails
- Typical pattern:
  - Application reads from replicas, writes to primary

Write Bottlenecks and Sharding

- Even with read replicas:
  - The primary can become a write bottleneck
- Options:
  - Optimize queries and indexes
  - Use connection pooling and caching
  - **Sharding** (splitting data across multiple primaries)
- Sharding challenges:
  - Cross-shard joins
  - Rebalancing shards
  - Managing transactions across multiple nodes

Modern Database Scaling Options

- Managed relational databases:
  - Amazon RDS, Cloud SQL, Azure Database
  - Provide read replicas, automated backups, failover
- Distributed SQL (preview only):
  - CockroachDB, YugabyteDB
  - Aim to provide horizontal scaling with SQL semantics
- Key-value stores:
  - Redis, DynamoDB, etc.
  - Designed for horizontal scaling from the start

Monitoring, Observability & Auto-Scaling Signals

- You cannot scale effectively without **observability**
- Need visibility into:
  - System metrics
  - Application performance
  - User-perceived latency

Key Metrics for Scaling Decisions

- Infrastructure:
  - CPU, memory, disk usage
- Application:
  - Requests per second
  - Error rate (4xx, 5xx)
  - Latency percentiles (p95, p99)
- Capacity:
  - Queue length
  - Thread/connection pool saturation
- Scaling triggers: CPU > 70% for 5 minutes; Average latency > 500 ms

Monitoring Tools and Pipelines

- Open-source stack:
  - **Prometheus** for metrics collection
  - **Grafana** for dashboards and alerts
- Cloud-native:
  - AWS CloudWatch Metrics and Alarms
  - GCP Cloud Monitoring
- Integration:
  - Auto-scaling rules connected to metrics
  - Alerts sent to Slack, email, or on-call systems

Preview: Auto-Scaling in Kubernetes

- Horizontal Pod Autoscaler (HPA):
  - Adjusts number of Pods based on metrics
- Simple flow:
  - Metrics server collects CPU usage
  - HPA controller compares with desired utilization
  - If average CPU > target: Increase Pod count
  - If below: Decrease Pod count
- Similar ideas apply to:
  - EC2 ASG
  - Serverless platforms

Best Practices for Scalable Architectures

- Practical guidelines to design for:
  - Scalability
  - Reliability
  - Security
  - Operability

Best Practice: Build Stateless Services

- Put persistent state into:
  - Databases
  - Caches
  - Object storage
- Avoid:
  - Local file sessions
  - Per-node in-memory mutable state
- Benefits:
  - Any instance can handle any request
  - Easy to add/remove instances behind load balancer

Best Practice: Use External Cache and Queues

- Caching:
  - Redis / Memcached to reduce database load
  - Store frequently read data and computed results
- Queues:
  - RabbitMQ, Kafka, SQS for asynchronous processing
  - Decouple user-facing requests from heavy background tasks
- Impact:
  - Smoother load profile
  - Improved user-perceived performance

Best Practice: Centralized Storage and CDN

- Store assets (images, videos, documents) in:
  - S3 or similar object storage
- Use a CDN:
  - CloudFront, Fastly, Cloudflare
- Benefits:
  - Reduced load on origin servers
  - Lower latency for global users
  - Easier scaling of static content delivery

Best Practice: Health Checks and Graceful Shutdown

- Health checks:
  - HTTP endpoints like `/health` and `/ready`
  - Load balancers and orchestrators rely on them
- Graceful shutdown:
  - Handle termination signals (SIGTERM)
  - Stop accepting new requests
  - Finish in-flight requests

```
process.on("SIGTERM", () => {
  server.close(() => {
    process.exit(0);
  });
});
```

Best Practice: Security and Least Privilege

- Avoid running services as **root**:
  - Use non-root users in Docker images
- Network security:
  - Restrict inbound ports to only what is needed
  - Place backends in private subnets
- Logging:
  - Keep centralized logs (e.g. ELK, Loki)
  - Logs help in **incident investigation** and attack detection
- Principle:
  - Each component should have only the permissions it truly needs

Best Practice: Logging and Metrics by Default

- Application-level logging:
  - Log structured JSON events
  - Include correlation IDs for distributed tracing
- Metrics:
  - Expose `/metrics` endpoint for Prometheus
  - Track per-endpoint latency and error rate
- This observability enables:
  - Confident scaling
  - Capacity planning
  - Fast root-cause analysis

Summary and Looking Ahead

- Today you learned:
  - Foundations of scaling and bottlenecks
  - Vertical vs horizontal scaling and why load balancing is crucial
  - How to design stateless, scalable web APIs
  - Practical load balancing with Docker, Nginx, and AWS
  - Database and observability considerations
- Next steps:
  - Apply these ideas in labs
  - Explore Kubernetes Ingress and service meshes for advanced traffic control